Today

Welcome to your next lesson: Class-level basics: attributes.

I'm preparing your personalized lesson based on what you've learned so far, what comes next, and the best ways to help things click.

Step by step, you’re moving forward.

Imagine we have a Pet class. Every pet you create from it (Dog, Cat, etc.) is an object.

Now suppose you want to store one fact that is true for all pets, like the word "pet" itself — not separately on each dog or cat, but once for the whole Pet type. That “one shared fact on the class” is what we’re going to learn about.

Quick check for you, Ron 🧠

If you had a Pet class and 3 pet objects (pet1, pet2, pet3), which sounds more like shared data?

A fact like “all of these are pets”
Or each pet’s own name

You can just answer in words: which one feels shared across all of them?

The fact that they’re all pets

Nice, exactly 👍

Look at the visual below 👇

The big Pet box has a single label kind = "pet" inside it. Underneath, you see pet1, pet2, pet3 boxes, each pointing to

that same kind = "pet".

This new idea is:

A class attribute is data stored once on the class, and all instances share it.

In your own words: what is that kind = "pet" label showing — is it something stored on each pet object separately, or something stored once on the Pet class that they all share?

Class vs instances with shared class attribute

something stored they all share

Yes, perfect — something stored once that they all share 🎯

In Python, that “shared on the class” thing looks like this:

Python
```
class Pet:
    kind = "pet"   # shared class data 👈

pet1 = Pet()
pet2 = Pet()

print(Pet.kind)   # access through the class
print(pet1.kind)  # access through an object
print(pet2.kind)  # access through another object
```
You don’t need to run this yet — just look at it.

Step by step:

kind = "pet" is written inside the class Pet: block
It is not inside a method, not assigned to self
That makes kind a class attribute: shared data stored on Pet itself

Make sense so far? 🙂

If yes, I’ll give you a tiny task where you spot the class attribute yourself.

Class vs instances with shared class attribute

yes it makes sense inside a function is local to that funcion so woulnt be available to all and its not tagged self (This instances attrirbute) so its globally shared to all instances, this one i didt know despite having written a few thousand plus page python oop apps. ive gone outside the class for globals good to know

I like how you connected that to what you already know about locals and self — that’s exactly the right mental model 💪

Check out the visual just below: one class Pet box holds kind = "pet", and pet1, pet2, pet3 all “point” to that same kind.

Your turn, super quick 🌟

Looking at that diagram:

Is kind = "pet":

stored three times (once inside each of pet1, pet2, pet3), or stored once on Pet and shared by pet1, pet2, and pet3?

Answer in your own words, just one short sentence.

Class vs instances with a shared class attribute

stored once on the parent class Pet and accessable to all instances

What do you think? Pick the option that’s the shared class attribute 👇

In this Python code, which name is the class attribute (shared data stored on the class)?

python

```
class Pet:
    kind = "pet"
```

```
pet1 = Pet()
pet2 = Pet()

print(pet1.kind)
```

A.
kind

B.
pet1

C.
Pet()

D.
pet1.kind

Exactly right 🎉

kind is the class attribute because it’s defined directly inside the class Pet: body, not inside a method and not on self.
pet1 and pet2 are instances (objects created from the class).
pet1.kind is how an instance accesses that shared class attribute, but the actual shared data lives on the class as Pet.kind.

So you’ve nailed the key idea:

A class attribute is shared data stored on the class, written directly in the class body.

Let’s lock it in with a tiny hands-on step 🧪

Open the Python code editor and write this (or paste and run it):

Python

```
class Pet:
    kind = "pet"

pet1 = Pet()
pet2 = Pet()

print(Pet.kind)
print(pet1.kind)
print(pet2.kind)
```
Run it, then hit Submit code.

I'll check it and we'll do one playful final question after that 😄

```
Python
class Pet:
    kind = "pet"

pet1 = Pet()
pet2 = Pet()

print(Pet.kind)
print(pet1.kind)
print(pet2.kind)
```
Nice work running that, Ron 💪

You just saw that:

Pet.kind prints "pet"
pet1.kind prints "pet"
pet2.kind prints "pet"

All three are using the same shared class attribute kind stored on Pet 🐾

One last fun check, then we're done with this idea 🎯

Imagine this code:

```
Python
class Car:
    wheels = 4

car1 = Car()
```

```
car2 = Car()

print(car1.wheels)
```

Which of these is the class attribute (the shared data stored on the class), not an object or an expression that uses it?

A. wheels

B. car1

C. Car()

D. car1.wheels

Just answer with a letter 🙂

a

Correct again ✅ A is right.

wheels is the class attribute because it’s defined directly inside class Car:.
car1 / car2 are instances.
car1.wheels is just how an object accesses that shared wheels value.

You’ve clearly got the key idea:

A class attribute is shared data stored once on the class and can be reached through the class or any instance.

You’ve mastered the goal for this lesson 🎯

Ready to wrap up this lesson and move on to the next one?

Yes, let’s move on

Awesome work today, Ron 🎯

Here’s what you locked in:

A class attribute is data written directly in the class body (like kind = "pet" or wheels = 4).
That data is stored once on the class and is shared by all instances (Pet.kind, pet1.kind, pet2.kind all point to the same value).

You’re thinking about scope, self, and shared data in exactly the right way — that will pay off a lot as your OOP code grows 💻

Whenever you’re ready, you can jump into the next lesson, and I’ll keep building on this with you step by step.

```
class Pet:
    kind = "pet"

pet1 = Pet()
pet2 = Pet()

print(Pet.kind)
print(pet1.kind)
print(pet2.kind)
```